

Shortest Path: Dijkstra's Algorithm

(ADA 2022, CSE 222)

Diptapriyo Majumdar

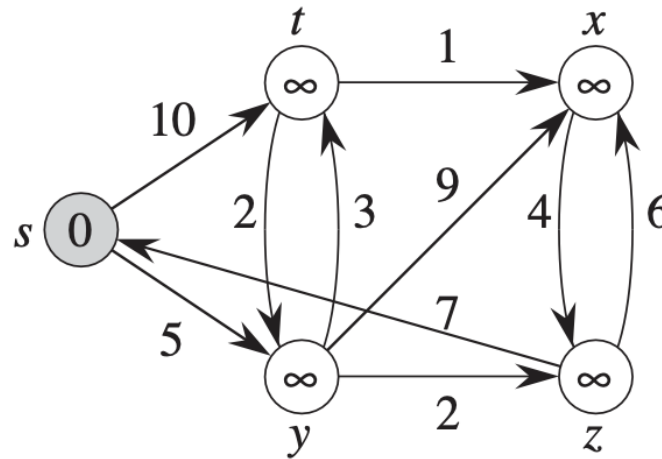
March 23, 2023

Outline

- ① Algorithm to compute Shortest Path
- ② Proof of Correctness
- ③ Data Structure for Implementation

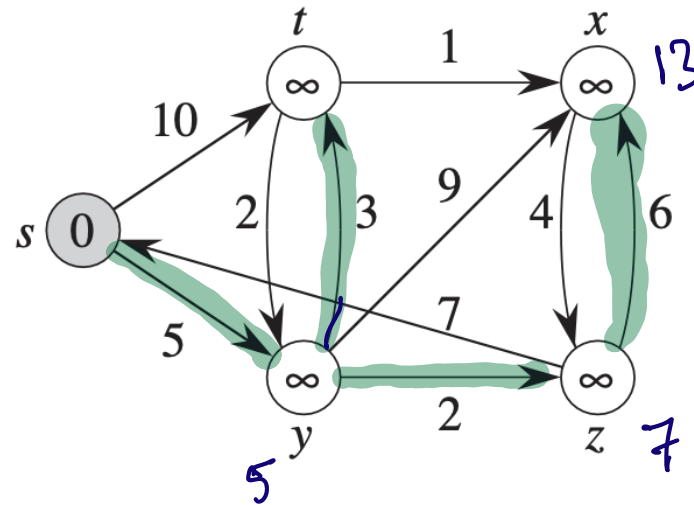
Shortest Path

- **Input:** A directed graph $G = (V, E)$, a weight function $\text{weight} : E(G) \rightarrow \mathbb{R}^+$ and $s \in V(G)$.
- For a vertex $u \in V(G)$, an s - u path is a directed path from s to u in G .
- **Goal:** For all $u \in V(G)$, compute the weight of a shortest s - u path in G .



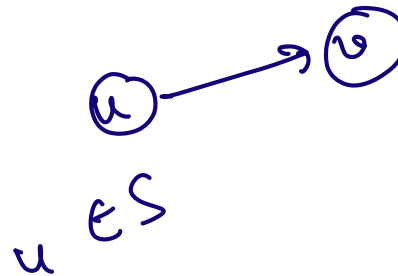
Some possible attempt

- Start from the vertex s .
- Choose a vertex where you can reach by a shortest weighted edge and so on.



High Level Idea

- For every $v \in V(G)$, maintain a vector $d[v]$ and maintain a set S .
- Initialize $d[s] \leftarrow 0$, $S \leftarrow \{s\}$ and for all $v \in V(G) \setminus \{s\}$, initialize $d[v] \leftarrow \infty$.
- Repeat the following step while there is an edge from S to $V \setminus S$.
 - $(u, v) \leftarrow$ an edge with minimum $d[u] + \text{weight}(u, v)$.
 - $S \leftarrow S \cup \{v\}$.



Algorithm

Initialize $d[s] \leftarrow 0$;

For all $v \in V(G) \setminus \{s\}$, $d[v] \leftarrow \infty$;

$S \leftarrow \emptyset$ and $Q \leftarrow V(G)$; $\Rightarrow$

while $Q \neq \emptyset$ **do**

$u \leftarrow \underline{\text{ExtractMin}(Q)}$; $\rightarrow$ greedy
 $S \leftarrow S \cup \{u\}$;

for each $v \in N_G(u)$ **do**

if $d[v] > d[u] + \text{weight}(u, v)$ **then**

$d[v] = d[u] + \text{weight}(u, v)$;

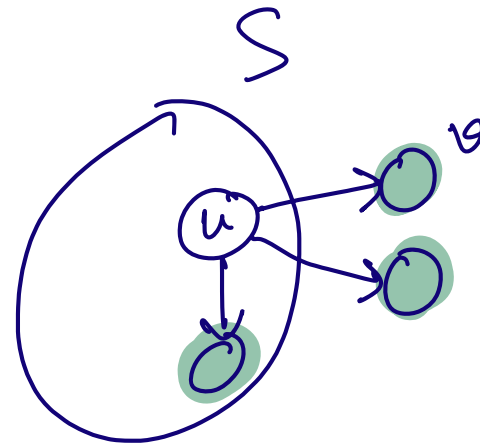
$\text{prev}(v) \leftarrow u$;

end

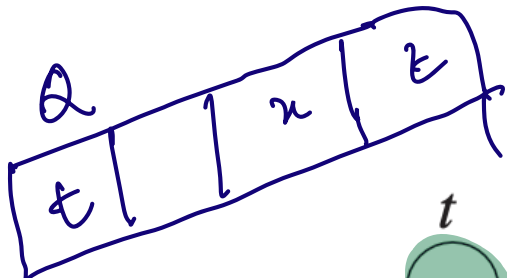
end

end

$Q \rightarrow$ Binary Heap data structure.

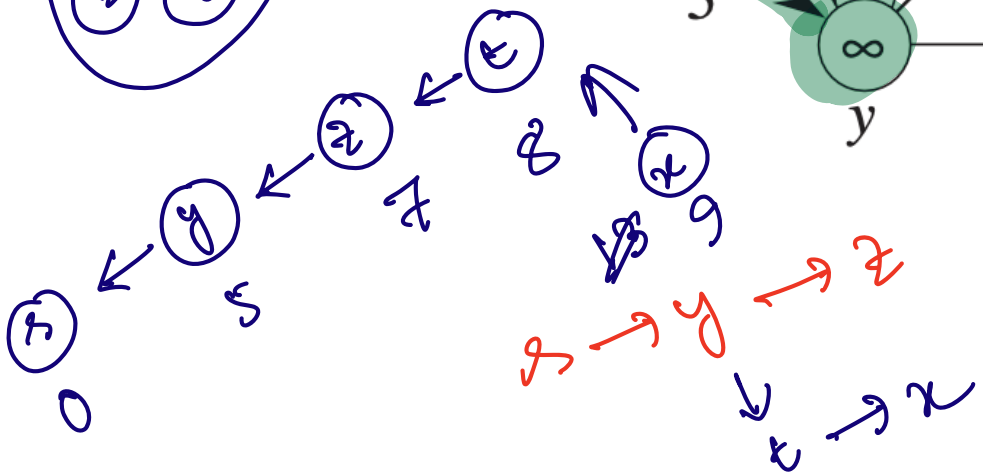
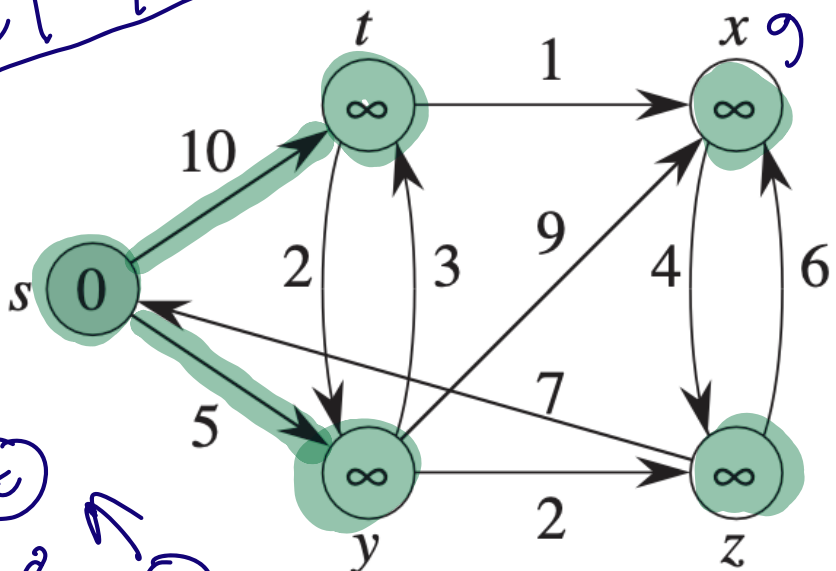
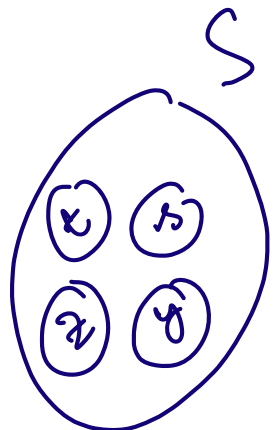


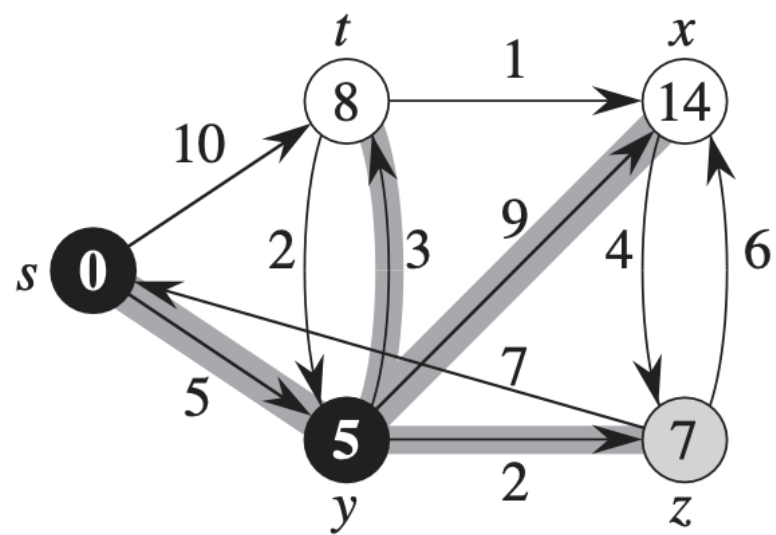
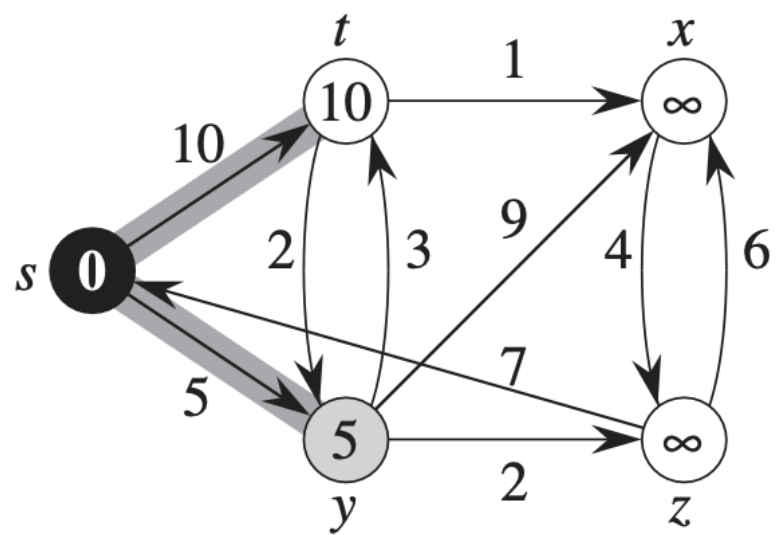
Q

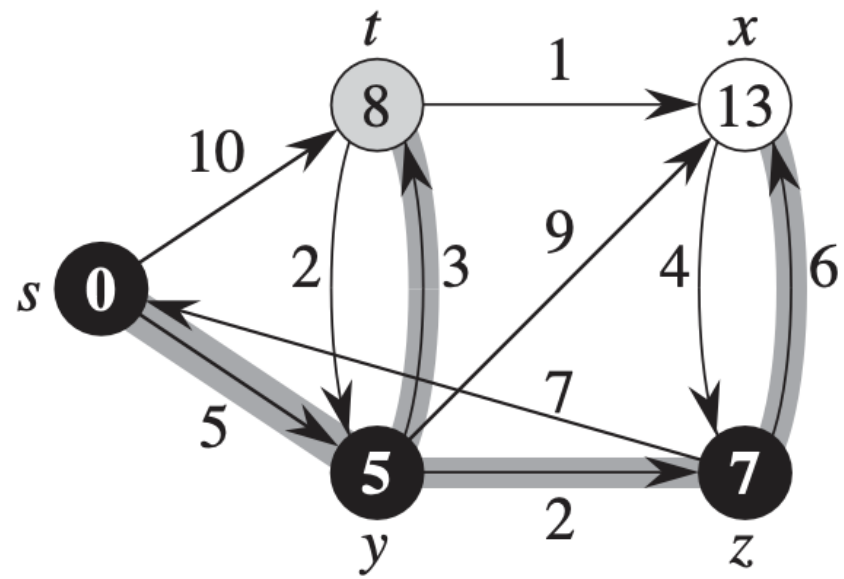


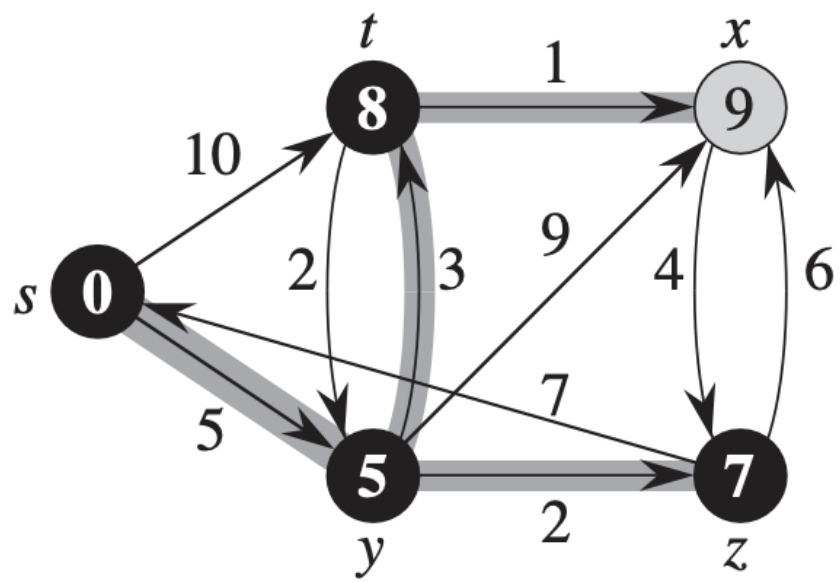
Illustration

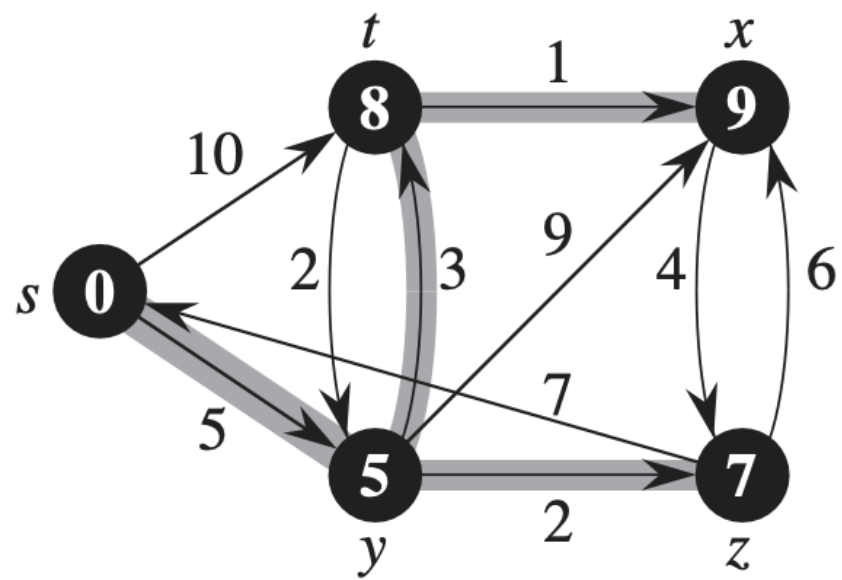
	d
s	0 ✓
t	∞ 10 8 ✓
y	∞ 5 ✓
x	∞ 14 13 9
z	∞ 7 ✓











Outline

- ① Algorithm to compute Shortest Path
- ② Proof of Correctness
- ③ Data Structure for Implementation

Correctness Proof (sketch)

- Let $\delta(s, u)$ be the weight of a shortest path from s to u .
- Key idea:** Once a vertex u is added to S , $d[u]$ must contain the value of $\delta(s, u)$.
 $d[u] = \delta(s, u)$

Proof: By induction on $|S|$.

Base Case: $|S|=0$ or $S = \emptyset$.

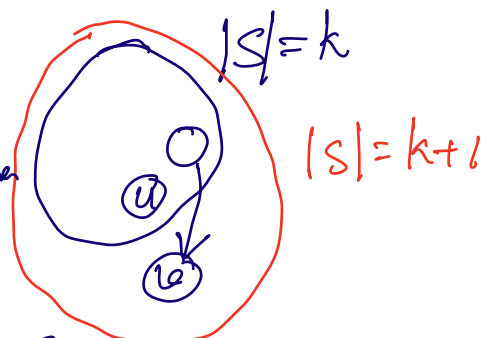
$$|S|=1. \quad S = \{s\} \quad d[s] = 0 = \delta(s, s)$$

Induction Hypothesis: The statement is true for
all $|S| \leq k$.

Case $|S| = k+1$.

Let v be the vertex added by Dijkstra's Algorithm
at $(k+1)$ th iteration.

Let v be the $(k+1)$ -th vertex added to S by Dijkstra Algorithm.



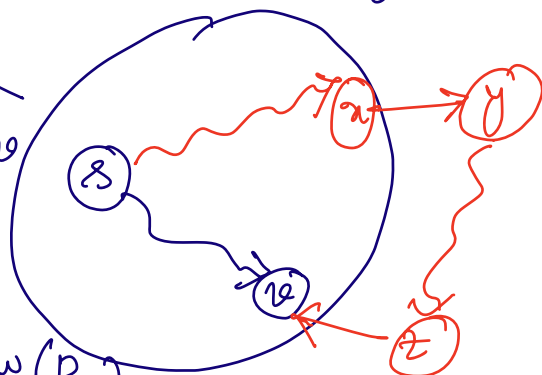
Then it must be that $d[v] \leq d[w]$ for all

$$w \in V \setminus S$$

Let P_v be the path chosen by algorithm

But consider some other shortest path $\hat{P}_v$ from s to v in G .

Assume that $w(\hat{P}_v) < w(P_v)$



Assumption is P_v is not shortest path from s to v .

x was added ^{to S} by algorithm before adding v to S .

$d[x] = \delta(s, x) \rightarrow$ by induction hypothesis.

$$d[x] \leq d[v]$$

when x was added to S , then (x, y) is an edge $d[y]$ was computed

$$\delta(s, y) \leq d[y] \leq d[x] + w(x, y)$$

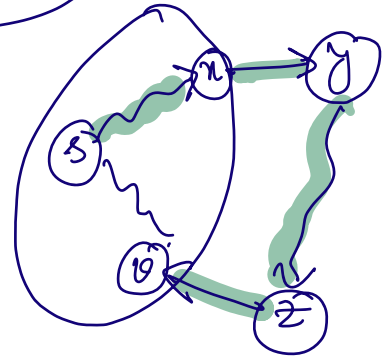
But y was not added to S after x .

In particular v was also added to S before y .

then, $d[v] \leq d[y]$

$$\delta(s, y) \leq \delta(s, v)$$

Since $\hat{P}_v$ is a shortest path from s to v ,



y appears before v in $\hat{P}_v$.

also all edges have nonnegative

Hence $\delta(s, y) \leq \delta(s, v)$.

$$\begin{aligned} d[v] \leq d[y] &\leq d[x] + w(x, y) \\ &\leq d[v] + w(x, y) \end{aligned}$$

Then $d[y] < d[v]$

Outline

- ① Algorithm to compute Shortest Path
- ② Proof of Correctness
- ③ Data Structure for Implementation

Algorithm

$O(mn)$
 $O(|V||E|)$

Q
Min-Heap

$O(|E| + |V|^2)$

Initialize $d[s] \leftarrow 0$;

For all $v \in V(G) \setminus \{s\}$, $d[v] \leftarrow \infty$;

$S \leftarrow \emptyset$ and $Q \leftarrow V(G)$;

while $Q \neq \emptyset$ **do** $O(n)$

$u \leftarrow \text{ExtractMin}(Q)$; $O(\log n)$

$S \leftarrow S \cup \{u\}$;

for each $v \in N_G(u)$ **do**

if $d[v] > d[u] + \text{weight}(u, v)$ **then**

$d[v] = d[u] + \text{weight}(u, v)$;

$\text{prev}(v) \leftarrow u$;

end

end

end

} $\text{deg}(u)$
Fibonacci Heap

use
 $O(m + n \log n)$